

→ Spank

→ Big data

↳ Very large size data

(Peta byte, Exabyte)

||

$$TB = 10^{12} B$$

$$PB = 10^{15} B$$

$$EB = 10^{18} B$$

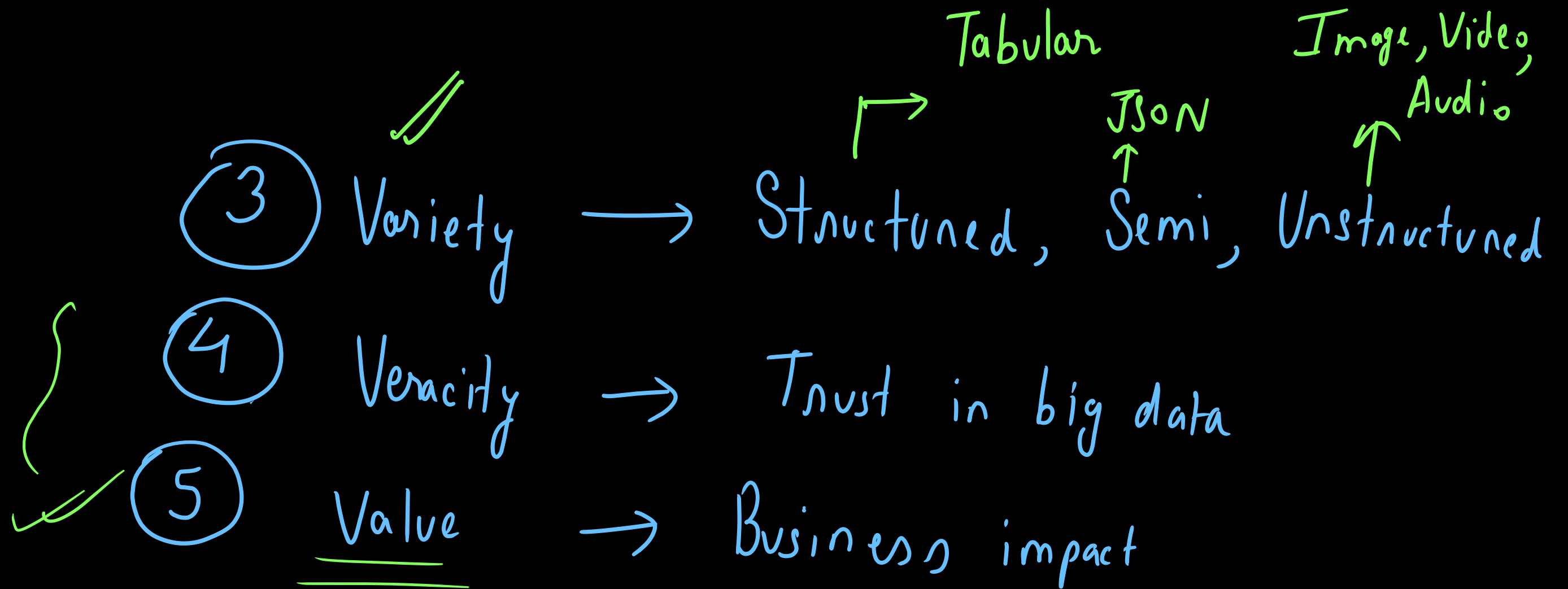
→ 5 V's of big data

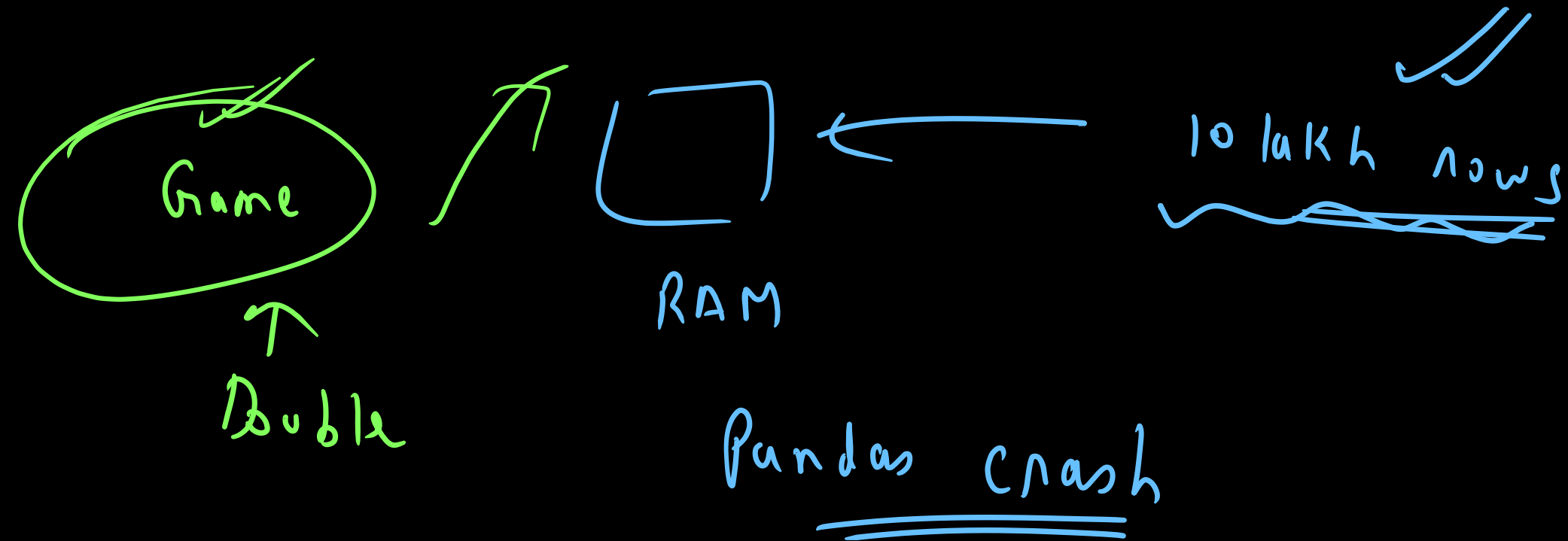
① Volume → Amount of data (PB, EB, TB)

② Velocity → Speed at which big data is generated

Google Maps ↔ Stream processing (Realtime)

Salary prepared by HR. ↔ Batch mode





Pandas

1. Memory constraint : Load entire dataset into RAM
2. I/O operation : Reading & writing large files are very slow & require compute
3. Lack Parallelism : Cannot take full advantage of Multi-processing.

Spark

↳ Very very fast

↳ Distributed data processing

↳ Can handle big data

Pandas

→ Slow

→ No distributed data processing

→ Cannot handle

→ Spark can be written in :

1> Scala (Primarily)

2> Java

3> R

4> Python (PySpark)

→ Spark has following capability:

1) Spark SQL

✓ 2) Spark MLlib

3) Spark GraphX

4) Spark Streaming (Realtime data)

Pandas

↳ filter

↳ select

Pyspark

Don't execute immediately

↳ Transformation → filter, select, groupby

↳ Action

Computation (Execution) is performed

& return result

↳ show()

↳ count()

Spark is Lazy in Nature

inplace = True

↓
Not executed

DAG

✓ Original data

✓ filter (age > 26)

✓ select (name)

✓ Show

↳ Execution is performed

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("LazyEvalExample").getOrCreate()

data = [("Alice", 25), ("Bob", 30), ("Cathy", 29)]
df = spark.createDataFrame(data, ["name", "age"])

# Transformations (LAZY)
df_filtered = df.filter(df.age > 26)
df_selected = df_filtered.select("name")

# No computation happens till now!
df_selected

DataFrame[name: string]

df_selected.show()

+-----+
| name |
```

Span K

↳ In-memory processing

↳ RAM

↳ lazy Evaluation (optimization)

→ Break until 22:25 PM.

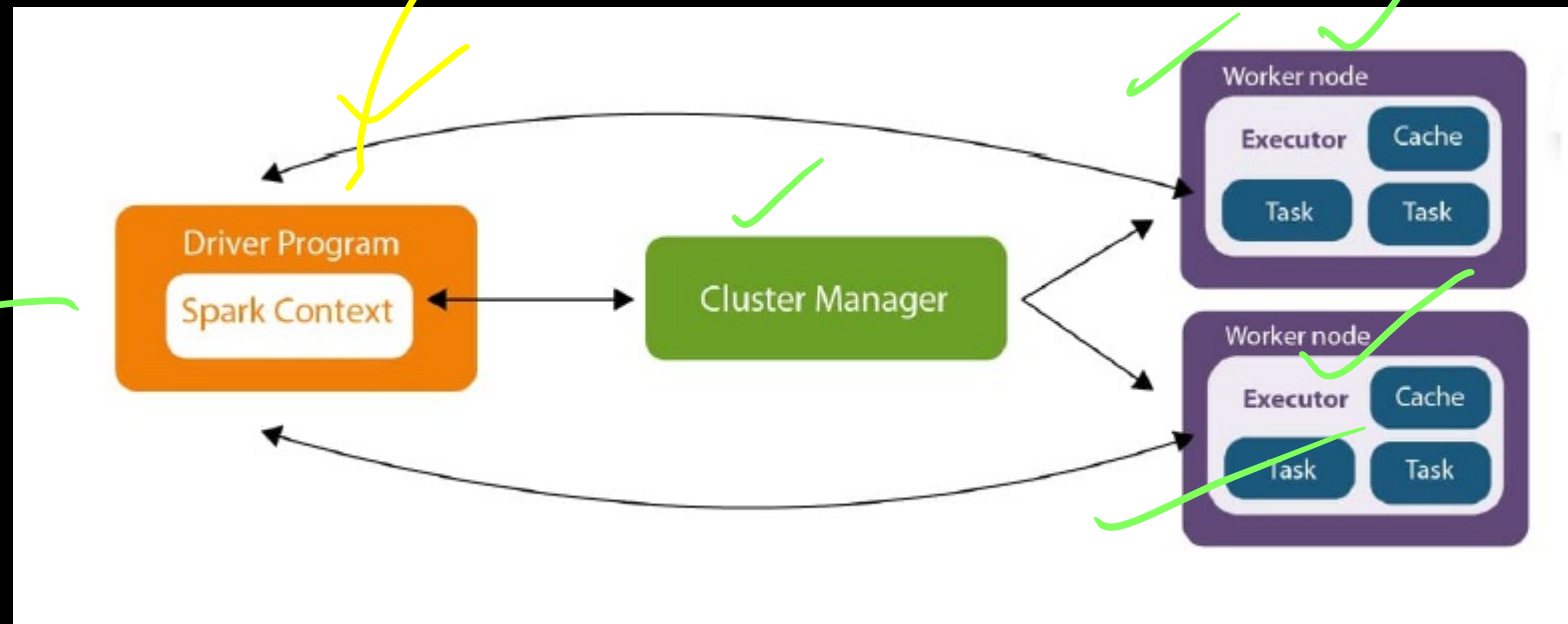
- 1> Driver
- 2> Cluster Manager
- 3> Worker Nodes

(CPU, RAM)

```
# Create a Spark Session  
spark = SparkSession.builder.master("local[*]").getOrCreate()
```

Spark Session / Spark Context

Provide Compute



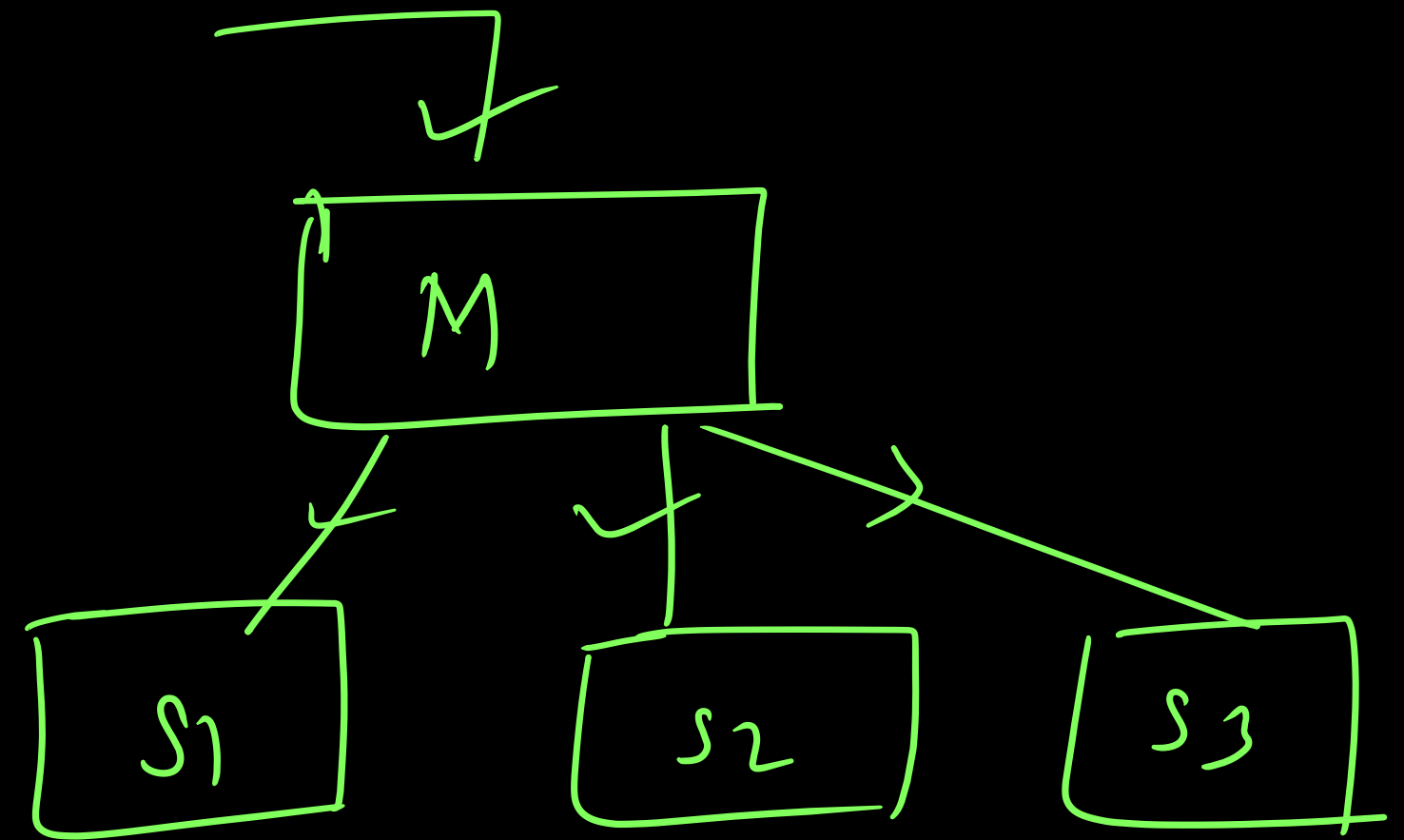
DAG



Multiple Task

✓
1. Driver will create DAG

2. These will be divided into multiple task



3. Task are sent to worker nodes

4. Worker nodes executes task in parallel

5. Return result to driver

Dataframe

- High level
- Performance optimized
- Faster

RDD (Resilient distributed dataset)

- Low level control
- Not optimized performance
- Slower

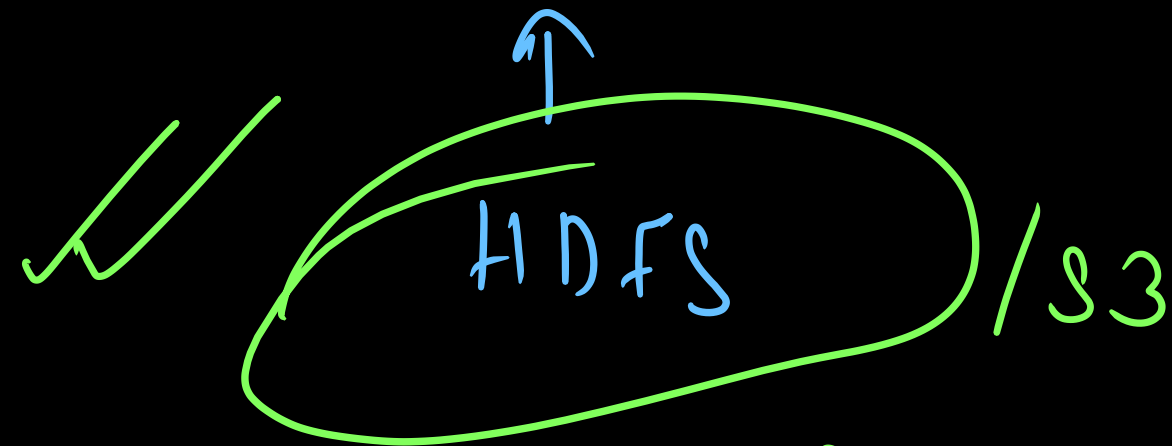
Resilient Distributed dataset (RDD)

- ↳ Cannot handle structured data. (like csv, parquet)
- ↳ Only operate on text file

→ Fault - tolerant

Data was split into
partition. These are
processed independently
& parallelly

Map Reduce / Spark (compute)



DB

→ Schema on
Write

Datalake

→ Schema on
Read

FAANG

Data Analyst /
Data Scientist

Applied
Scientist

/SW+ML

5 interview

→

→

→

→

→

1 Googlyness / Mining Manager ✓

1/2 Data Structure & Algorithm

1 ML System Design

2 rounds of ML Depth

